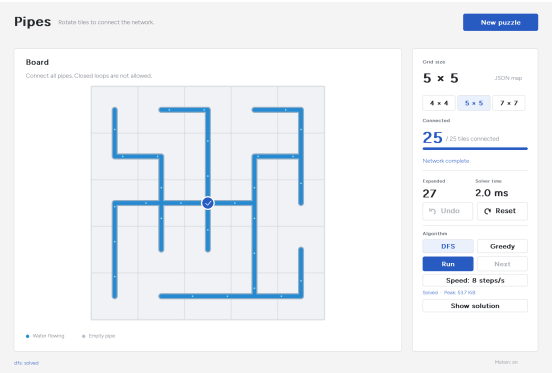
[UNIVERSITY]
[FACULTY / DEPARTMENT]

ASSIGNMENT 1 — SEARCH

Semester 1, academic year 2026–2027

Course: Introduction to Artificial Intelligence

Pipes & Light Up:
Implementing Search Algorithms for Two Logic Puzzles



Group members

Full name	Student ID	Class
[Name 1]	[ID 1]	[Class 1]
[Name 2]	[ID 2]	[Class 2]
[Name 3]	[ID 3]	[Class 3]

Instructor: [Instructor name]

[City], 31 October 2026

Contents

1 Introduction **2**

1.1 Objectives . . . 2
1.2 The two chosen problems . . . 2
1.3 Environment and tooling . . . 2
1.4 Repository layout . . . 2
1.5 How we studied and implemented each problem . . . 3

2 Problem 1: Pipes **3**

2.1 Studying the assignment and the rules . . . 3
2.2 Search problem formulation . . . 4
2.3 Search space and shared pruning . . . 4
2.4 Blind search: Depth-First Search . . . 5
2.5 Heuristic search: Greedy Best-First . . . 5
2.6 Implementation and visual demo . . . 6
2.7 Creating inputs . . . 6
2.8 Experimental results: time and memory . . . 6
2.9 Limitations . . . 8

3 Problem 2: Light Up (Akari) **8**

3.1 Research process and rules . . . 8
3.2 Problem model . . . 8
3.3 Difficulty design and puzzle generation . . . 9
3.4 Algorithm selection and implementation . . . 9
3.5 Demonstration and experimental protocol . . . 10
3.6 Measured results . . . 11

4 Conclusion **12**

1 Introduction

1.1 Objectives

Assignment 1 asks the group to pick two problems from the Logic Puzzles at https://www.puzzle-pipes.com/ (and related problems on that site) and implement, in Python 3, two search algorithms for **each** problem: one *blind* search (DFS or BrFS) and one *heuristic* algorithm. The submission must include a step-by-step visual demo (game style), and a report describing how the group studied and implemented each problem, with tables of execution time and memory consumption per algorithm per input, and an interpretation of those numbers.

1.2 The two chosen problems

- **Pipes** (https://www.puzzle-pipes.com/): rotate pipe tiles to form one connected, leak-free, acyclic network. Algorithms: **DFS** (blind) and **Greedy Best-First** (heuristic).
- **Light Up / Akari** (https://www.puzzle-light-up.com/, part of the same Logic Puzzles family): place bulbs on white cells so that every white cell is illuminated, no two bulbs see each other, and every numbered wall has exactly its count of adjacent bulbs. Algorithms: **DFS** (blind) and **Backtracking + MRV + CP** (heuristic constraint search).

Each experimental comparison uses paired inputs under identical resource limits. The updated Light Up experiment measures DFS and CSP on 60 distinct seeded boards; Pipes retains its previously published evidence.

1.3 Environment and tooling

- **Language**: Python 3 (mandatory per the assignment). Development uses Python 3.13 with `uv` for dependency management.
- **Graphics and demo**: `pygame-ce` for both games; `pygame-gui` adds controls to the Light Up interface.
- **Testing and code quality**: `unittest` across both packages, `ruff` linting, GitHub Actions CI.
- **Measurement**: `time.perf_counter` for wall time; `tracemalloc` for peak Python allocations.
- **Documents**: LaTeX (tectonic) for this report and the slides.

1.4 Repository layout

One shared repository, one package per puzzle:

```
Introduction-to-AI/
  main.py              Common launcher: uv run main.py [pipes|lightup]
  pipes_problem/       Pipes game, DFS/Greedy, UI, benchmark, tests
  light_up_problem/    Light Up, DFS/CSP, UI, benchmark, tests
  docs/                Algorithm analysis, benchmark evidence (CSV + md)
```

```
  report/             LaTeX sources of this report (main-vi, main-en)
  slides/             LaTeX sources of the slides (main-vi, main-en)
```

Run instructions are in the repository `README.md`. Launch either game:

```
uv run main.py pipes
uv run main.py lightup
```

1.5 How we studied and implemented each problem

The group followed the same process for both puzzles:

1. **Learn the rules**: play the original puzzle on its official site, note the invariants (win condition, constraints), reference `puzzle-pipes.com` / `puzzle-light-up.com`.
2. **Formulate as search**: define state, initial state, actions, transition model, goal test and costs; choose an efficient state representation (orientation prefixes for Pipes, bitmasks for Light Up).
3. **Design algorithms**: one blind search (DFS) and one heuristic search (Greedy / MRV with constraint propagation), plus sound pruning rules; write pseudocode first.
4. **Implement and demo**: build the pygame game with a step-by-step search visualization and solution replay.
5. **Create inputs**: write a seeded, witness-checked puzzle generator; document the provenance of every input.
6. **Measure**: run automated benchmarks recording time, memory and node counts to CSV; store evidence under `docs/benchmarks/`.
7. **Analyze**: explain the numbers, compare theory with measurement, state limitations honestly.

2 Problem 1: Pipes

2.1 Studying the assignment and the rules

We studied Pipes by playing the original game at https://www.puzzle-pipes.com/ and observing its behavior. Playing first, we distilled the rules as follows:

- The board is an $m \times m$ grid of pipe tiles. Each tile has a *fixed shape* (straight, elbow, T-junction, cross, end) and a changeable *orientation*; rotating a tile is the only move.
- Each shape exposes ports on some of its North–East–South–West sides. Rotation never changes the port count, only directions.
- Goal: every open port must meet a reciprocal open port on the adjacent tile (no port leaves the board, no dangling port), all tiles must form **one connected** network, and the network must contain **no cycle**.

- The water source shown in the original game is a visualization anchor: once the network is complete, water flows from the source to every tile. Moving the source does not change the set of valid boards.

In textbook terms this is a single-agent, deterministic, fully observable, discrete, static task. It is a *satisfaction* problem: any valid board is a solution; minimizing manual rotations is not required (the original game does not score rotations either).

2.2 Search problem formulation

Let $n = m^2$ and number the tiles in row-major order. Tile i has a fixed shape and an orientation domain R_i of its distinct rotations. We formulate:

Component	Definition in this implementation
Input	Saved scrambled board (shapes + orientations), grid size, source
State	Prefix $s = (a_0, \ldots, a_{k-1})$ of assigned orientations
Initial state	Empty prefix $()$, with all tile shapes known
Action	Assign a boundary-valid orientation to tile k (row-major order)
Transition	$s' = s + (a_k)$, accepted only after all feasibility checks
Goal test	$k = n$ and the complete board satisfies every rule
Cost	1 per assignment; neither solver ranks by cost
Solution	One complete valid orientation tuple

Table 1: Search formulation for Pipes.

One subtlety: a *solver step* assigns an absolute orientation to the next tile, while a *player step* rotates a tile by 90 degrees. One assignment can equal zero to three clicks. Both solvers share the assignment model, which makes their comparison fair. The saved scrambled board only supplies the domains R_i and the rotation order; search always starts from the empty prefix and never reads a stored solution.

2.3 Search space and shared pruning

The number of complete assignments before boundary checks is $U = \prod_i |R_i| \leq 4^n$: elbows, T-junctions and ends have 4 distinct rotations, straights 2, crosses 1. A typical 4×4 board has $U = 4^{16} \approx 4.29 \times 10^9$; the 7×7 upper bound is $4^{49} \approx 3.17 \times 10^{29}$. The assignment tree has depth n, branching factor at most 4, and **every prefix has exactly one parent** (fixed assignment order), so no closed set is needed for duplicate detection.

Orientations pointing off the board are removed up front. For each candidate child, the shared `compatible()` routine applies three sound pruning layers to both solvers:

1. **Match assigned neighbors**: every unassigned domain R_i is restricted to orientations matching the assigned neighbors' ports (present and absent alike). An empty domain rejects the branch.

2. **Cycle rejection**: if assigned tiles already form a cycle, reject (disjoint-set check).

3. **Possible connectivity**: build an *optimistic* graph over tiles — an edge exists if some remaining orientation could open toward the other tile. If this graph cannot connect every tile, reject.

At the root we check the port count: a tree over n vertices has exactly $n-1$ edges $= 2(n-1)$ ports, and rotation preserves the count, so a different total proves unsolvability immediately (the converse does not hold). All three layers only remove states that cannot lead to a solution, so completeness is preserved. DFS is therefore uninformed in its *traversal order* but is a constraint-pruned search, not a blind rotation enumerator.

2.4 Blind search: Depth-First Search

DFS follows the first feasible child until it reaches a goal or a dead end, then backtracks. Rotations are enumerated in each tile's saved-orientation order — no heuristic information is used:

```
SEARCH_DFS(input):
  build boundary-valid domains R_i
  if root feasibility fails: return UNSOLVABLE
  return VISIT(())

VISIT(s):
  if budget exhausted: return LIMIT_REACHED
  count one expansion
  if GOAL(s): return SOLVED(s)
  for a in boundary-valid rotations of tile |s|:
    child = s + (a)
    if FEASIBLE(child):           # 3 pruning layers
      r = VISIT(child)
      if r is SOLVED or LIMIT_REACHED: return r
  return UNSOLVABLE
```

The prefix space is finite and acyclic and the pruning is sound, so DFS is *complete* given enough resources. The first solution found does not minimize assignments. Worst-case time is exponential in n; memory keeps the current path plus pending alternatives — this Python implementation stores copied prefix tuples and generator frames, about $O(n^2)$ along one branch.

2.5 Heuristic search: Greedy Best-First

Greedy always pops from a min-heap the state with the fewest *remaining choices*:

$$h(s) = \sum_{i \text{ unassigned}} \log_2 |D_i(s)|, \qquad \text{priority}(s) = (h(s),\ -\text{depth}(s),\ \text{insertion order})$$

where $D_i(s)$ is the boundary-valid domain of tile i after matching assigned neighbors. The sum is the logarithm of the product of remaining domain sizes: domains $(1, 2, 4)$ give $h = 0+1+2 = 3$, while $(1, 1, 2)$ give $h = 1$ — state B is less "free" and is preferred. We chose this heuristic because (i) it is cheap, reusing domains already computed during pruning, and (ii) every solution at the same depth needs exactly n assignments, so $n-k$ distinguishes nothing, while the log-domain sum prioritizes branches that narrow fastest. h is *not admissible* — it measures remaining uncertainty, not a cost bound — and no g term is used.

```
SEARCH_GREEDY(input):
  build boundary-valid domains; root feasibility check
  push () into min-priority queue with priority 0
  while queue is not empty:
```

```
  if budget exhausted: return LIMIT_REACHED
  s = pop minimum-priority state;  count one expansion
  if GOAL(s): return SOLVED(s)
  for a in boundary-valid rotations of tile |s|:
    child = s + (a);  D = FEASIBLE(child)
    if D exists: push child with (h(child), -|child|, serial)
return UNSOLVABLE
```

On the finite, acyclic prefix space, keeping all feasible candidates makes Greedy complete given enough resources; but Greedy must hold its frontier in the heap, so memory can grow exponentially.

2.6 Implementation and visual demo

The implementation lives in the `pipes_problem` package: `model.py` (rules, validation, board generation), `search.py` (both solvers, pruning, timing and memory), `app.py` (pygame interface), `worker.py` (isolated measurement process), `benchmark.py` (batch runner). The demo interface offers:

- Manual play: click to rotate (Shift for counterclockwise), right-click to lock, Undo/Reset; connected pipes carry animated blue water from the source.
- **Step-by-step search demo**: pick DFS or Greedy, then *Run / Pause / Next / Speed*. Each event is labeled: *expand* (DFS visits / Greedy pops a prefix), *assign* (DFS chooses a child), *queue* (Greedy pushes), *reject* (a pruned orientation — tile outlined in red with the reason), *backtrack*. *Next* separates expansion from successor generation.
- *Show solution* replays the final assignment on its own; the board returns to its saved state before a run. *Unsolvable* and *limit reached* are distinct, explicit messages.

2.7 Creating inputs

Inputs are produced by the project generator (not transcribed from the web): nine default maps covering sizes $4 \times 4, 5 \times 5, 7 \times 7$ with seeds 261–263, stored as compact JSON (`size`, `source`, `tiles`) with the filename as the input ID. In addition, `maps/custom/cdai.json` is a hand-edited *unsolvable* map used to check messaging and exhausted-space behavior. Runtime generation by `--size`/`--seed` is also available. These are the team's own examples (not official website puzzle IDs), and the generator never stores a solution in the map file.

2.8 Experimental results: time and memory

Protocol. Runs were executed on Linux x86_64 (kernel 6.8), Python 3.13.14, pygame-ce 2.5.8. Both algorithms receive the *same* nine maps, the same row-major tile order, the same pruning, a 30-second limit and 200,000 expansions per run. Each (map, algorithm) pair runs 5 times: time is a headless `perf_counter` solve (no drawing); memory is the peak Python allocation measured by `tracemalloc` in a separate replay, combined only when both passes agree on status, returned board and expansion count. The full 90-run dataset is stored at `docs/benchmarks/2026-10-25-pipes.csv`; the tables below report medians per input.

Interpreting the results.

Input	Algorithm	Solved/runs	Time (ms)	Memory (KiB)
pipes-4x4-261	DFS	5/5	3.380	26.836
	Greedy	5/5	2.866	11.695
pipes-4x4-262	DFS	5/5	3.047	27.078
	Greedy	5/5	3.386	12.359
pipes-4x4-263	DFS	5/5	3.149	26.984
	Greedy	5/5	2.741	12.500
pipes-5x5-261	DFS	5/5	5.352	53.203
	Greedy	5/5	7.094	19.500
pipes-5x5-262	DFS	5/5	5.906	53.344
	Greedy	5/5	5.882	19.219
pipes-5x5-263	DFS	5/5	4.564	53.297
	Greedy	5/5	5.712	19.219
pipes-7x7-261	DFS	5/5	23.247	145.852
	Greedy	5/5	27.049	27.594
pipes-7x7-262	DFS	5/5	21.938	145.930
	Greedy	5/5	26.669	27.070
pipes-7x7-263	DFS	5/5	40.785	145.852
	Greedy	5/5	38.246	29.094

Table 2: Pipes — time and peak memory per input (median of 5 runs, measured 25 October 2026). All 90/90 runs are *solved*. Source: `docs/benchmarks/2026-10-25-pipes.csv`.

- **Time grows quickly with size** (3–5 ms at 4×4 to 22–41 ms at 7×7): variables grow from 16 to 49 and each successor's feasibility check touches more assigned neighbors, matching the exponential prediction for the search space.
- **DFS is faster than Greedy at 5×5 and 2 of 3 maps at 7×7.** On easy inputs the pruning already steers DFS's first branch almost straight to a solution; the priority queue only adds heap push/pop and h computation without changing traversal much. This shows that a heuristic is *not free*: its bookkeeping cost must be repaid by ranking quality.
- **On the hardest map (pipes-7x7-263) Greedy wins** (38.2 vs 40.8 ms): this map forces real backtracking, and ranking by "fewest remaining choices" explores tight branches first, saving some wasted probing. The margin is small (5%) but appears exactly where heuristics should help.
- **Memory: Greedy uses 4–5× less than DFS** (12–29 KiB vs 27–146 KiB). Counter to the textbook worry about Greedy's exponential frontier, the tracemalloc figure reflects this implementation's actual Python allocations: DFS copies a prefix tuple and holds a generator frame per recursion level, while Greedy's frontier stays tiny on easy inputs because pruning discards almost everything. On much harder inputs Greedy's frontier could still balloon — further experiments would be needed to confirm.
- **All 90 runs solved** within limits; no run hit the 30-second or 200k-expansion budget on this input set.

Size	Algorithm	Solved/runs	Median time (ms)	Median memory (KiB)
4×4	DFS	15/15	3.149	26.984
	Greedy	15/15	3.097	12.359
5×5	DFS	15/15	5.261	53.297
	Greedy	15/15	6.157	19.219
7×7	DFS	15/15	24.469	145.852
	Greedy	15/15	31.695	27.594

Table 3: Pipes — summary by board size (3 maps $\times$ 5 runs per figure).

2.9 Limitations

The default input set is quite easy under the current pruning, so neither solver is pushed to its limits; any "winner" claim holds only for this kind of input. We have not yet ablated the pruning layers separately, added many unsolvable maps, or tried an MRV heuristic. These limitations are recorded in `docs/pipes-analysis.md` as future work.

3 Problem 2: Light Up (Akari)

3.1 Research process and rules

Light Up, also called Akari, is a grid-based logic puzzle developed by Nikoli in 2001 [5]. The group started with Nikoli's official rules, examined the interactions on Puzzle Light Up, and then studied solving methods and difficulty assessment before implementing the program [5, 6]. McPhail proves that deciding whether a general Light Up instance has a solution is NP-complete, by a reduction from Circuit-SAT [1]. This motivates our use of search supported by pruning and constraint deductions.

A board contains white cells and black walls, some numbered 0–4. A valid solution satisfies Nikoli's rules [5]:

- Bulbs occupy white cells and illuminate their own cell plus uninterrupted row and column cells up to a wall or the boundary.
- Every white cell must be lit.
- No bulb may illuminate another. Bulbs may share a row or column when a wall separates them.
- A wall numbered k has exactly k orthogonally adjacent bulbs; diagonals do not count. Unnumbered walls only block light.

Puzzle Light Up [6] informed the size/level selectors, left-click placement and right-click $\times$ marks. The group added algorithm playback and search statistics for the educational demonstration.

3.2 Problem model

Let P be the white cells, with a binary variable x_i for each $i \in P$; $x_i = 1$ means a bulb is present. Let $V(i)$ contain cells visible from i, including itself; L be an uninterrupted horizontal or vertical

white segment; and $N(w)$ be the white neighbors of numbered wall w. The rules become:

$$\sum_{j \in V(i)} x_j \geq 1 \qquad \text{(every white cell is lit)},$$

$$\sum_{j \in L} x_j \leq 1 \qquad \text{(no bulbs see one another)},$$

$$\sum_{j \in N(w)} x_j = k_w \qquad \text{(exact numbered-wall count)}.$$

Initially no bulbs are placed; a goal satisfies all three constraint families. The code stores bulb, illuminated and legal-location sets as bitmasks. For CSP, a placed bulb has domain $\{1\}$, a forbidden cell $\{0\}$, and an undecided cell $\{0, 1\}$. A separate goal validator is shared by solvers, generated-board verification and experimental-result validation.

3.3 Difficulty design and puzzle generation

The group consulted Dekker [2], who measures difficulty through the solving techniques required, from wall/dark-cell deductions to patterns and guesses. To simplify this assignment, the program **does not implement that rating system**: easy/normal/hard differ only in their **target wall density**. In `light_up_problem/generator/generate.py`, a board of side n receives $M = \text{round}(\rho n^2)$ walls:

Level	ρ	7×7	10×10	14×14	25×25
easy	16%	8	16	31	100
normal	10.5%	5	10	21	66
hard	5.5%	3	6	11	34

M counts numbered and unnumbered walls together; the table follows Python's rounding exactly. All levels use the same generation procedure:

1. Select M random wall positions using the seed.
2. Repeatedly place a bulb on a randomly chosen dark white cell until all white cells are lit, producing a witness solution.
3. Derive wall numbers from adjacent witness bulbs, leave approximately 25% of walls unnumbered, and validate the witness before returning the board.

The solver does not receive the witness. Clue-density and constraint-coverage metrics are recorded as metadata, not used to assign difficulty labels. A hard label therefore does not guarantee a longer solve time than easy, and solution uniqueness is not checked.

3.4 Algorithm selection and implementation

DFS baseline. DFS selects the first dark cell in row-major order and tries legal illuminators in fixed order. It descends immediately after the first feasible candidate, returning to another candidate only when the subtree fails. Feasibility checks reject conflicts, unreachable wall counts and dark cells without an illuminator. Later siblings exclude earlier options to avoid generating the same bulb sets in different placement orders. Explicit stack frames avoid Python recursion limits; parent links reconstruct the solution.

CSP: Backtracking + MRV + Constraint Propagation. The group consulted Pulles's combination of forced deductions and backtracking in Sections 4.1.1 and 4.1.3 [3], together with `tracksolver.py` in Borroot/akari [4]. These informed the handling of dark cells, candidates and state restoration. The current program implements this approach with bitmasks, propagation at each branch and **dynamic constraint-level MRV**; the following describes the actual code.

Constraint propagation (CP) repeats until no domain changes:

- A satisfied wall forbids its undecided neighbors. A wall whose deficit equals its available neighbors forces bulbs at all those neighbors.
- A dark cell with one legal illuminator forces that bulb. Each new bulb also forbids other bulbs along its illuminated lines.
- Diagonal deductions forbid positions that would leave too few available neighbors for a wall; a contradiction rejects the branch.

After CP, MRV (Minimum Remaining Values) selects the constraint with the fewest choices: $\binom{u}{r}$ combinations for a wall with u available neighbors and deficit r, or the legal-illuminator count for a dark cell. Walls win ties. Conflicting combinations are discarded; values are tried by decreasing gain in illuminated dark cells.

```
BACKTRACK_MRV_CP(state):
  check budgets; propagate until no change
  if contradiction: return failure
  if goal: return solution
  choose constraint with fewest remaining choices
  for choice ordered by lighting gain:
    child = apply choice to current checkpoint
    result = BACKTRACK_MRV_CP(child)
    if solved or limited: return result
    restore parent checkpoint before next choice
  return failure
```

The pseudocode expresses the backtracking flow; the code uses explicit stack frames. A checkpoint restores both bulbs and legal domains. The function `backtracking_mrv_cp` propagates at the root as well as after each choice and stops at the first solution. Neither DFS nor CSP guarantees minimum bulb count. A CSP node can contain several forced bulbs, so nodes are not equal units of work: the benchmark measures the combined MRV, CP and value-ordering implementation.

3.5 Demonstration and experimental protocol

The interface supports manual play or solver selection, Start Search, Pause/Resume and arrow-based history review at 2/8/30 steps per second. CSP shows each deduction, marks forbidden cells with $\times$ and restores states on backtracking. A separate worker keeps the UI responsive. Visualization disables the timeout while retaining node/memory limits; benchmarks disable trace/animation and retain all three limits.

The 31 October 2026 experiment uses 4 sizes $\times$ 3 levels $\times$ 5 distinct seeds: 60 boards and 120 DFS/CSP measurements. Each board is run once per solver; all runs are sequential in fresh processes, alternating solver order across successive seeds. Both solvers use these seed ranges:

Size	easy	normal	hard
7	70097–70101	70194–70198	70291–70295
10	100097–100101	100194–100198	100291–100295
14	140097–140101	140194–140198	140291–140295
25	250097–250101	250194–250198	250291–250295

Both solvers receive **30 s, 500,000 expanded nodes and 512 MiB estimated memory**. `TIMEOUT`, `NODE_LIMIT`, `MEMORY_LIMIT` and `UNSOLVABLE` remain distinct; budgets are not automatically extended. UNSOLVABLE requires a proven contradiction or exhaustive branch failure, rather than reaching a resource limit. Runtime is the solver's `perf_counter` elapsed time with `tracemalloc` active, excluding generation, process startup and external validation. Memory is traced peak Python allocation, distinct from RSS and the estimate used to stop search. All SOLVED outputs are independently validated. Cooperative checkpoint checks can slightly overshoot 30 s.

The raw CSV, summary and manifest record seeds, exact JSON boards, hashes and environment under `docs/benchmarks/`, with prefix `2026-10-31-lightup-dfs-csp`. The following tables use this dataset.

3.6 Measured results

Algorithm	SOLVED	TIMEOUT	NODE_LIMIT	MEMORY_LIMIT
DFS	52	8	0	0
CSP	60	0	0	0

Size	Difficulty	Alg.	Solved	T/N/M	Time (s)	Peak (KiB)	Nodes	Bulbs
7	easy	DFS	5/5	0/0/0	0.0075	24.9	11	9
7	easy	CSP	5/5	0/0/0	0.0066	25.2	4	9
7	normal	DFS	5/5	0/0/0	0.0066	26.8	9	7
7	normal	CSP	5/5	0/0/0	0.0103	29.0	6	7
7	hard	DFS	5/5	0/0/0	0.0074	27.3	8	7
7	hard	CSP	5/5	0/0/0	0.0107	30.5	6	6
10	easy	DFS	5/5	0/0/0	0.0229	55.9	27	15
10	easy	CSP	5/5	0/0/0	0.0288	56.2	9	14
10	normal	DFS	5/5	0/0/0	0.0218	60.0	15	12
10	normal	CSP	5/5	0/0/0	0.0309	61.8	8	11
10	hard	DFS	5/5	0/0/0	0.0229	62.4	12	11
10	hard	CSP	5/5	0/0/0	0.0314	67.8	10	10
14	easy	DFS	5/5	0/0/0	0.2330	135.3	261	28
14	easy	CSP	5/5	0/0/0	0.0801	109.1	12	25
14	normal	DFS	5/5	0/0/0	0.0724	117.3	34	22
14	normal	CSP	5/5	0/0/0	0.1030	123.5	17	19
14	hard	DFS	5/5	0/0/0	0.0832	126.1	22	19
14	hard	CSP	5/5	0/0/0	0.1168	134.6	15	16
25	easy	DFS	2/5	3/0/0	30.0001	2237.0	9 328	73
25	easy	CSP	5/5	0/0/0	0.9558	562.1	31	68
25	normal	DFS	1/5	4/0/0	30.0004	3583.3	13 886	58
25	normal	CSP	5/5	0/0/0	1.0274	696.9	33	52
25	hard	DFS	4/5	1/0/0	0.7835	870.3	46	40.5
25	hard	CSP	5/5	0/0/0	1.0421	868.0	34	35

Table 4: Medians across five distinct seeds per configuration. T/N/M count TIMEOUT, NODE_LIMIT and MEMORY_LIMIT separately. Attempt time includes stopped runs, not their completion times. Bulbs is the median bulb count over SOLVED runs only; the Solved column gives its sample count.

Time, memory and node medians include all five attempts, including limited runs. Bulbs includes SOLVED runs only, with sample counts in the Solved column; a TIMEOUT is not zero bulbs. Different solved counts can mean different seed subsets. The summary also reports solved-only time medians.

Evaluation. DFS solves **52/60** boards and CSP **60/60**; all eight TIMEOUT runs belong to DFS. No run reaches NODE_LIMIT or MEMORY_LIMIT, but both caps remain enabled. Among 52 jointly solved pairs, CSP is faster on 15 and the median $t_{DFS}/t_{CSP} = 0.73$; CSP is therefore not uniformly faster. Small well-pruned boards can favor DFS because CP costs more per node, while CSP improves coverage of larger branching cases within the same budgets. Bulb counts describe the returned solutions, not proven optima. The 25 October measurements use the older generator and are not pooled here.

Five seeds per configuration are a small sample; wall-density difficulty is not calibrated and uniqueness is unchecked. Next steps are broader inputs, unsolvable boards, CP/MRV ablations and uninstrumented timing runs.

4 Conclusion

The repository provides visual search demonstrations for both puzzles. Pipes compares DFS with Greedy Best-First; the updated Light Up experiment compares DFS with Backtracking + MRV + CP. The new measurements use the current generator and five distinct seeds per configuration; earlier runs are archived separately.

- **Heuristics have a cost.** Constraint propagation can reduce search while costing more per node. On small well-pruned Light Up maps, DFS can be faster; the current CSP comparison also improves coverage on larger branching cases.
- **Keep comparisons paired.** Five different seeds per configuration sample input diversity; they are not five repetitions of one board. Both algorithms receive identical maps and time/node/estimated-memory budgets.
- **Limits are outcomes, not unsolvability.** TIMEOUT, NODE_LIMIT and MEMORY_LIMIT are reported separately. Stopped-attempt durations do not estimate completed solve times. See the raw CSV and the Light Up results table.
- **Measurement definitions matter.** Traced Python allocation is not RSS or the memory-limit estimate; CSP and DFS nodes do not represent equal work.

Future work: more seeds and unsolvable maps, CP/MRV/pruning ablations, uniqueness checking, and separate runtime measurements without tracemalloc instrumentation.

References

[1] B. McPhail. *Light Up is NP-complete*, 2005. https://www.mountainvistasoft.com/docs/lightup-is-np-complete.pdf

[2] V. Dekker. *A Difficulty Measure for Light Up Puzzles*. Bachelor thesis, Leiden University, 2015. https://theses.liacs.nl/pdf/2014-2015Victor-Dekker.pdf

[3] B. Pulles. *Analysis of Akari*. Bachelor thesis, Radboud University, 2021; Sections 4.1.1 and 4.1.3. https://www.cs.ru.nl/bachelors-theses/2021/Bram_Pulles___1015194___Analysis_of_Akari.pdf

[4] Borroot. *akari*: solver implementation, particularly `src/tracksolver.py`. https://github.com/borroot/akari

[5] Nikoli. *Akari*: official rules. https://www.nikoli.co.jp/en/puzzles/akari/

[6] Puzzle Light Up. Online game and interface reference. https://www.puzzle-light-up.com/

[7] Puzzle Pipes. https://www.puzzle-pipes.com/

[8] S. J. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, Pearson, 2021 (Chapter 3: Searching; Chapter 6: Constraint Satisfaction Problems).

[9] Python 3 documentation: `time`, `tracemalloc`, `multiprocessing`, `heapq`, `unittest`. https://docs.python.org/3/

[10] pygame-ce documentation. https://pyga.me/docs/ pygame_gui documentation. https://pygame-gui.readthedocs.io/